

# Lab11 SSA Quad to ARM Assembly

## 1. 实验目标

形成一个 advDFG 图，用于指令选择；

在 advDFG 图上，进行指令选择（使用课堂上说的一个 ARMv7-A 指令子集），优化指令的使用，形成 preSchedule。

在 preSchedule 的基础上，进行指令线性化，处理好 jump/Cjump, Phi, 以及 Function entrance, function call, 以及 return。最后形成 schedule 结构，并输出至 .s 文件。

## 2. 数据结构

### 2.1 assem.hh

```
struct AssemInstr
```

1. assem 是模板字符串
2. dst 是这条汇编定义了哪些 temp
3. src 是这条汇编使用了哪些 temp
4. jumps 是这条指令可能跳到哪些 label
5. label 用于标签指令

### 2.2 advDFG.hh

1. advDFGprog 管整个程序，里面是一组 advDFGfunc
2. advDFGfunc 管一个函数，里面是一组 advDFGblock
3. advDFGblock 管一个 basic block，里面有一个 advDFG
4. advDFG 真正的图，里面是一组 advDFGNode。它记录：这是入口、普通语句还是出口语句；它对哪条 QuadStm；它定义了哪个 temp；它使用了哪些 temp；它和哪些前驱/后继节点相连。

### 2.3 preSchedule.hh

结构也是三级：

1. preScheduleProg
2. preScheduleFunc
3. preScheduleBlock

preScheduleBlock 最关键，它里面有：

1. entryLabel 这个 block 的入口标签
2. phiFunctions 这个块开头的 phi 语句
3. selectedInstructions 这个块内部已经选出来的汇编
4. lastInstruction 这个块末尾那条关键 Quad，比如 JUMP、CJUMP、RETURN

## 2.4 schedule.hh

两层结构：

1. `ScheduleProg`
2. `ScheduleFunc`

这里已经没有 block 级结构了，说明到了这一步，basic block 已经被“线性化”了。也就是一个函数最后会变成一串顺序的 `linearizedInstructions`。

## 3. buildPreSchedule.cc

不负责真正“翻译”指令，只负责把 Quad 里的 block 整理成适合后面两个阶段使用的 preSchedule 结构。

1. 给每个函数建一个 `preScheduleFunc`
2. 给每个 basic block 建一个 `preScheduleBlock`
3. 把 block 里的语句拆分成三类存起来：
  - PHI 放进 `phiFunctions`
  - 普通语句留给后面的指令选择
  - block 最后那条控制流语句放进 `lastInstruction`

### 3.1 buildPreScheduleProg

遍历函数，再遍历 block，再遍历 block 里的语句。

遇到 PHI 就单独收集，其他语句先放到临时列表里，最后从后往前找出最后一条控制流语句。

## 4. buildAdvDFG.cc

instruction selection 在 tiling 或模板匹配时，需要先知道语句之间的依赖顺序。比如某条 LOAD 不能跑到它地址计算之前，某条 STORE 不能把依赖的值提前丢掉。有了图以后，就可以从图结构里找局部模式，而不只是机械地按原顺序扫一遍。

### 4.1 buildAdvDFGprog

输入是整个 `QuadProgram`，输出是整个程序的 advDFG 程序对象。

整体流程：

1. 遍历每个函数 func，对每个函数建一个 advDFGfunc，挂到程序里。
2. 遍历函数里的每个 block，对每个 block 建一个 advDFGblock，挂到函数里。
3. 为 block 建入口节点。入口节点类型是 EntryLabel，没有对应 Quad 语句，但它是这个块图的起点。
4. 初始化两个局部状态，lastTempDef 记录“某个 temp 最近是由哪个节点定义的”。previousNode 记录上一条处理过的节点，用来连顺序边。
5. 遍历 block 中的每条语句，如果是空语句或者 LABEL，就跳过。因为 LABEL 在这里不当成真正的运算节点，入口节点已经承担了块入口的角色。
6. 为语句建节点，节点类型根据 isExitLike 分成两种。普通语句：Statement；JUMP、CJUMP、RETURN：ExitStatement
7. 填节点信息

tempDefined 来自 firstDefinedTemp  
tempsUsed 来自 usedTemps  
chainDefined 用当前语句顺序号 chainIndex  
chainUsed 设成前一个节点的 chainDefined

8. 连顺序边：从前一个语句连到当前语句，代表块内语句按原顺序进行。
9. 连数据依赖边：对当前节点使用到的每个 temp，去 lastTempDef 里找最近一次定义这个 temp 的节点。找到了就加一条边：定义节点 -> 当前使用节点。如果当前节点定义了某个 temp，更新 lastTempDef。

## 5. selectInstr.cc

根据 advDFG，往 block 的 selectedInstructions 里塞具体 ARM 指令。

### 5.1 runInstructionSelectionPass

处理整个程序，遍历每个函数、每个 block，逐块调用 selectInstructionsForBlock。

### 5.2 selectInstructionsForBlock

处理一个 block。围绕 block 的 Quad 语句顺序做模板匹配和折叠。

遍历每个 statements，先尝试用 tryFoldPtrCalcMemoryAccess 做局部折叠，如果折叠失败，再交给 lowerStatement 按普通规则翻译。

### 5.3 tryFoldPtrCalcMemoryAccess

输入一整块语句、当前下标、当前 preScheduleBlock、临时编号计数器，以及一个输出参数 consumedUntil。

解决如下情况：先有一条 PTR\_CALC，算出一个临时地址，后面紧跟或者稍后出现一条 LOAD 或 STORE，并且这个地址临时在块内只被这一次使用。如果满足这些条件，它就不生成中间的 add 临时，而是直接把最终访存写成带偏移形式。步骤如下：

1. 先检查当前位置是不是 PTR\_CALC
2. 从 PTR\_CALC 里取出地址临时编号，也就是这个计算结果代表哪个 temp
3. 在后续语句里找唯一一个真正使用这个地址临时的 LOAD 或 STORE
4. 再检查这个地址临时在后续是否只用了一次
5. 如果可以折叠，就调用 emitFoldedLoadStore 生成最终的 ldr 或 str
6. 同时把 consumedUntil 设成被折叠掉的那条访存语句的位置，让外层扫描跳过它

### 5.4 lowerStatement

根据 QuadKind 分类处理。

1. MOVE  
如果源是 temp，就生成 mov dst, src。  
如果源是常量，就生成 mov dst, #imm，或者更大的常量用 movw 和 movt。  
如果源是名字，就生成 ldr dst, =label。
2. LOAD  
先把地址 term materialize 成一个 temp，然后生成 ldr dst, [addr]。
3. STORE  
先把 src 和 dst 都 materialize 成 temp，然后生成 str src, [dst]。
4. PTR\_CALC  
如果不能折叠，就单独生成 add。

常量偏移时优先直接 add。

更大的立即数会先 materialize 成临时，再做 add。

#### 5. MOVE\_BINOP

处理算术。如果右操作数是小常量，还会尽量用立即数形式，比如 `add dst, src, #1`。

#### 6. CALL 和 MOVE\_CALL

先把参数搬到 r0 到 r3，再 `bl` 或 `blx`，最后如果有返回值，把 r0 拷回目标 temp。

#### 7. EXTCALL 和 MOVE\_EXTCALL

和普通 `call` 类似，只是调用目标是外部函数名。

## 6. schedule.cc

---

它重点处理的是：

1. 块顺序
2. 条件跳转和无条件跳转
3. phi 的搬运
4. 函数序言和结尾

### 6.1 emitLoadImmediate

这个函数和 `select` 阶段里的类似，负责把一个整数常量装进寄存器。先把低 16 位用 `movw` 装进去，如果高 16 位不为 0，再用 `movt` 补上。

### 6.2 materializeTerm

把 `QuadTerm` 变成可以直接拿来用的 `tree::Temp*`：

1. 如果本来就是 `TEMP`，就直接返回那个 temp
2. 如果是 `CONST`，就新建一个临时，再用 `movw/movt` 把常量装进去
3. 如果是 `NAME`，就用 `ldr =label` 把地址装进去

`schedule` 阶段经常要插入一些“补充动作”，比如 phi 搬运前要先准备源操作数、分支条件前要准备比较值。这时候它就统一负责把操作数物化成 temp。

### 6.3 emitCallLike/emitExtCallLike

这两个函数分别处理：普通函数调用/外部函数调用

流程类似：

1. 先把参数放进 r0 到 r3
2. 发出调用指令
  - 普通函数用 `bl` 或 `blx`
  - 外部函数用 `bl`
3. 如果有返回值，把 r0 再搬回目标 temp

### 6.4 emitEpilogue

这是函数结尾固定模板。它做的事是恢复栈帧并返回：

1. 恢复 `sp`
2. 弹出保存的寄存器
3. `bx lr` 返回

也就是说，函数序言和结尾是 `schedule` 阶段统一插入的。

## 6.5 orderBlocks

给函数里的 basic block 排序，做一次 DFS 风格的遍历：

1. 从函数块列表里找一个起点
2. 如果块尾是 CJUMP，就优先按照 false / true 分支继续找后继块
3. 如果块尾是 JUMP，就沿跳转目标走
4. 如果有 exit\_labels，也继续把目标块纳入顺序

这个顺序决定了后面 schedule 时哪个分支可以做 fallthrough，哪个分支需要显式跳转。

## 6.6 emitParallelMoves

处理一组需要同时发生的赋值（主要针对 Phi 函数），尽量采用拓扑排序的方式进行赋值。比如 `t1=t2;t2=t3;`，就先进行 `t1=t2`，再进行 `t2=t3`，避免相互影响。如果存在环，则使用一个新的中间值。不过对于经过 SSA 处理的 Quad 语句，通常不应该出现环的情况，都应该可以正常处理。

## 6.7 scheduleProg

整体流程如下：

1. 遍历每个函数的 preSchedule
2. 为每个函数建一个 ScheduleFunc
3. 按 orderBlocks 的结果遍历 block
4. 给每个 block 输出口 label
5. 插入函数序言
6. 调用 `block->selectedInstructions` 并拼接该 block 的局部汇编
7. 根据块尾语句处理控制流
8. 最后得到整个函数的线性化结果

**块首：**

每到一个 block，schedule 会先输出这个 block 的 label。

如果这是函数的第一个 block，还会额外插入函数序言：

1. push 保存寄存器
2. 调整栈
3. 设置 fp
4. 搬运参数寄存器

**块尾：**

如果块尾是 RETURN：

1. 先把返回值物化到 r0
2. 再输出 epilogue
3. 最后 bx lr

如果块尾是无条件跳转：

1. 先对跳转边做 phi 搬运
2. 如果目标块不是紧跟着的下一个块，就显式发出 b 目标标签

如果目标块正好是下一块，就可以省掉这条跳转。

如果块尾是条件跳转：

1. 先把左右操作数物化成 temp

2. 发出 cmp
3. 尽量让 false 分支成为 fallthrough
4. 如果不行，就反向处理条件，或者插入一个中间 label

然后对 true/false 对应的边分别做 phi 搬运。

## 7. git graph

